



Module 1 — Introduction à Kubernetes

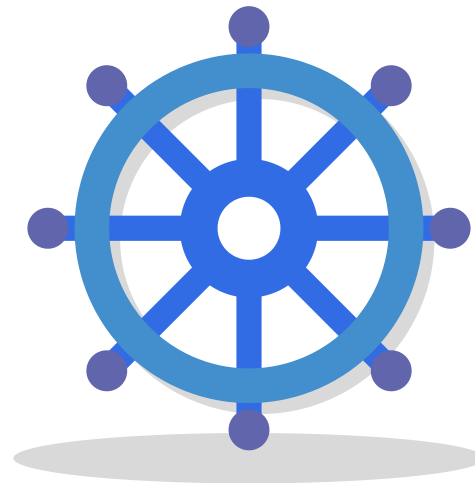
Origine, fonctionnalités et premiers objets

Kubernetes — Orchestration des conteneurs

Objectifs du module

À la fin de ce module, vous serez capable de :

- expliquer **quel problème** résout un orchestrateur ;
- situer Kubernetes : de Borg à la CNCF ;
- citer ses fonctionnalités et les runtimes (CRI) ;
- définir **Pod, label, sélecteur, contrôleur, Service** ;
- distinguer approche **impérative** et **déclarative** ;
- créer et interroger vos premiers objets avec `kubectl`.



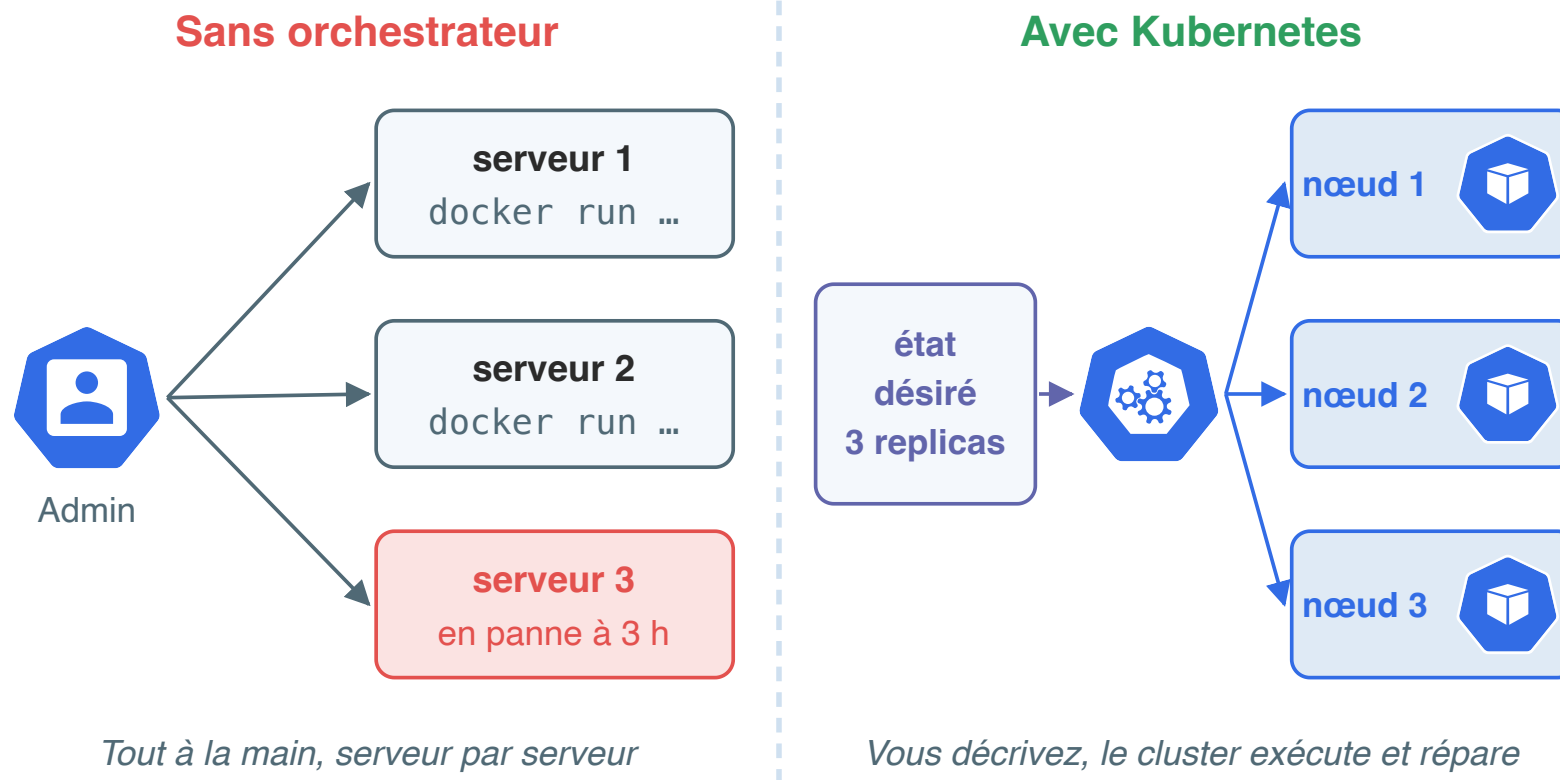
Plan du module

1. **Pourquoi Kubernetes ?** Le problème, l'origine, le rythme des versions.
2. **Fonctionnalités** : déploiement, auto-réparation, mise à l'échelle, rolling update, découverte de services.
3. **Runtimes et plateformes** : CRI, containerd, CRI-O, fin de dockershim, offres du marché.
4. **Composants et objets** : vue d'ensemble, Pods, labels, contrôleurs, Services.
5. **Déclaratif et kubectl** : premier contact avec le cluster.

Durée : 1 h 15, dont la démo 1 (premier Pod TrackR) et le TP 1 (trackr-front en recette).

1. Pourquoi Kubernetes ?

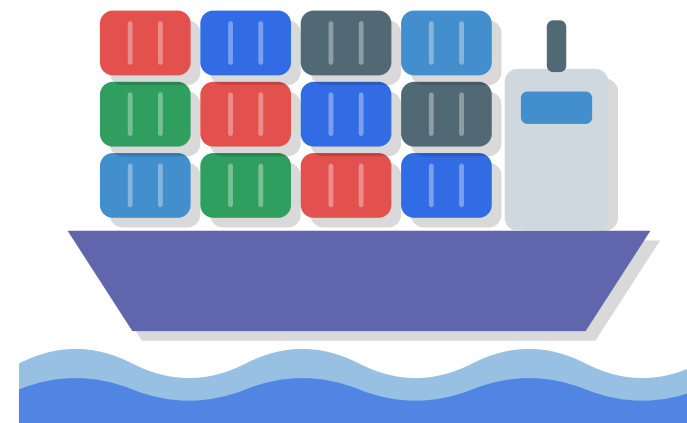
Le problème : des conteneurs à grande échelle



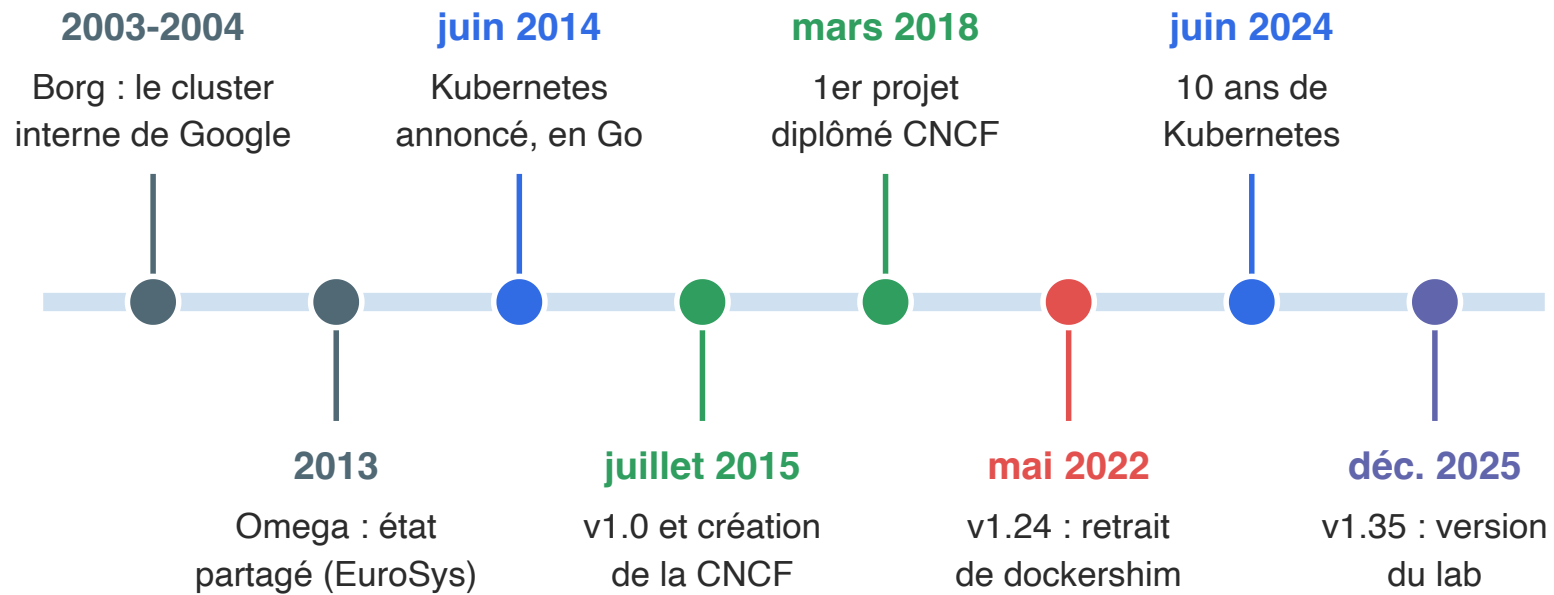
Qu'est-ce que Kubernetes ?

Kubernetes (du grec *kubernêtês*, « timonier »), abrégé **K8s**, est une plateforme open source d'**orchestration de conteneurs**.

- Vous décrivez l'**état désiré** : quelles images, combien de copies, quel réseau.
- Le cluster **place** les conteneurs sur les machines, les **surveille** et les **remplace**.
- Écrit en Go, sous licence Apache 2.0, gouverné par la **CNCF** (Cloud Native Computing Foundation).



De Borg à la CNCF



Google conçoit, la CNCF (Linux Foundation) gouverne

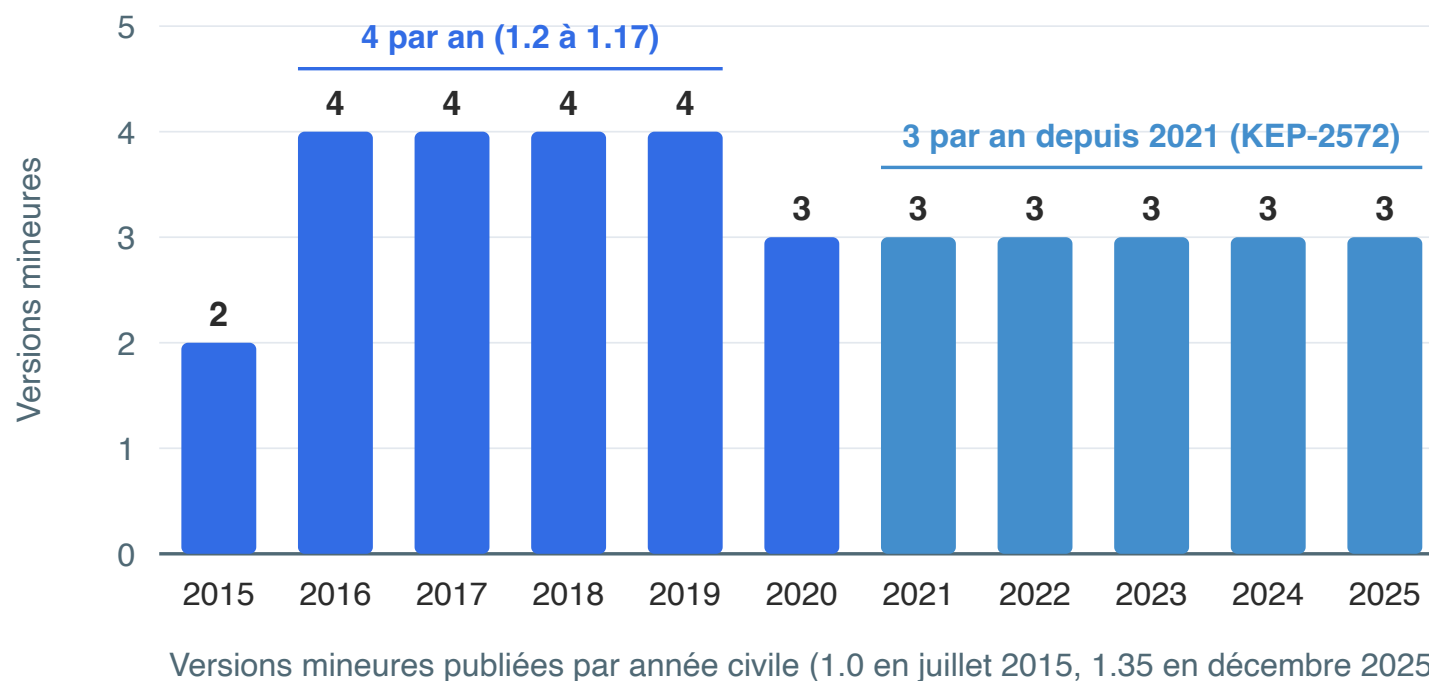
L'héritage de Borg et d'Omega

Kubernetes reprend dix ans d'exploitation de conteneurs chez Google :

- **Borg** regroupait des conteneurs coopérants dans une même unité (*alloc*) : c'est l'ancêtre du **Pod**.
- Les **labels** remplacent les noms de tâches rigides de Borg pour regrouper des objets.
- **Une adresse IP par Pod**, là où Borg partageait l'IP de la machine entre tâches.
- D'**Omega** vient l'idée d'un **état partagé** central, consulté par des contrôleurs indépendants.

Source : B. Burns, B. Grant, D. Oppenheimer, E. Brewer, J. Wilkes, « Borg, Omega, and Kubernetes », ACM Queue, 2016.

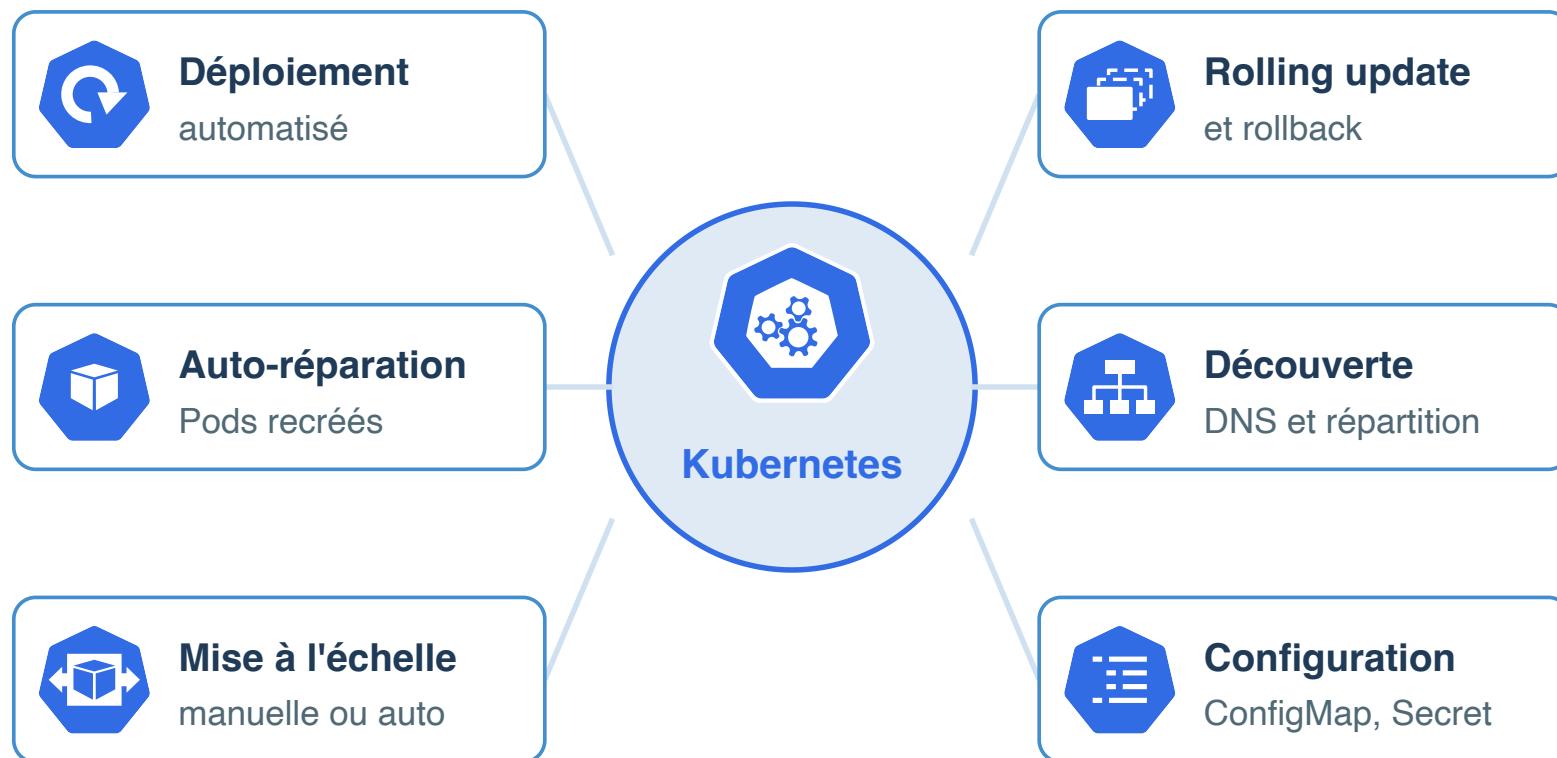
Trois versions mineures par an



Sources : kubernetes.io/releases ; blog Kubernetes « Release Cadence Change », juillet 2021.

2. Fonctionnalités

Ce que fait un orchestrateur

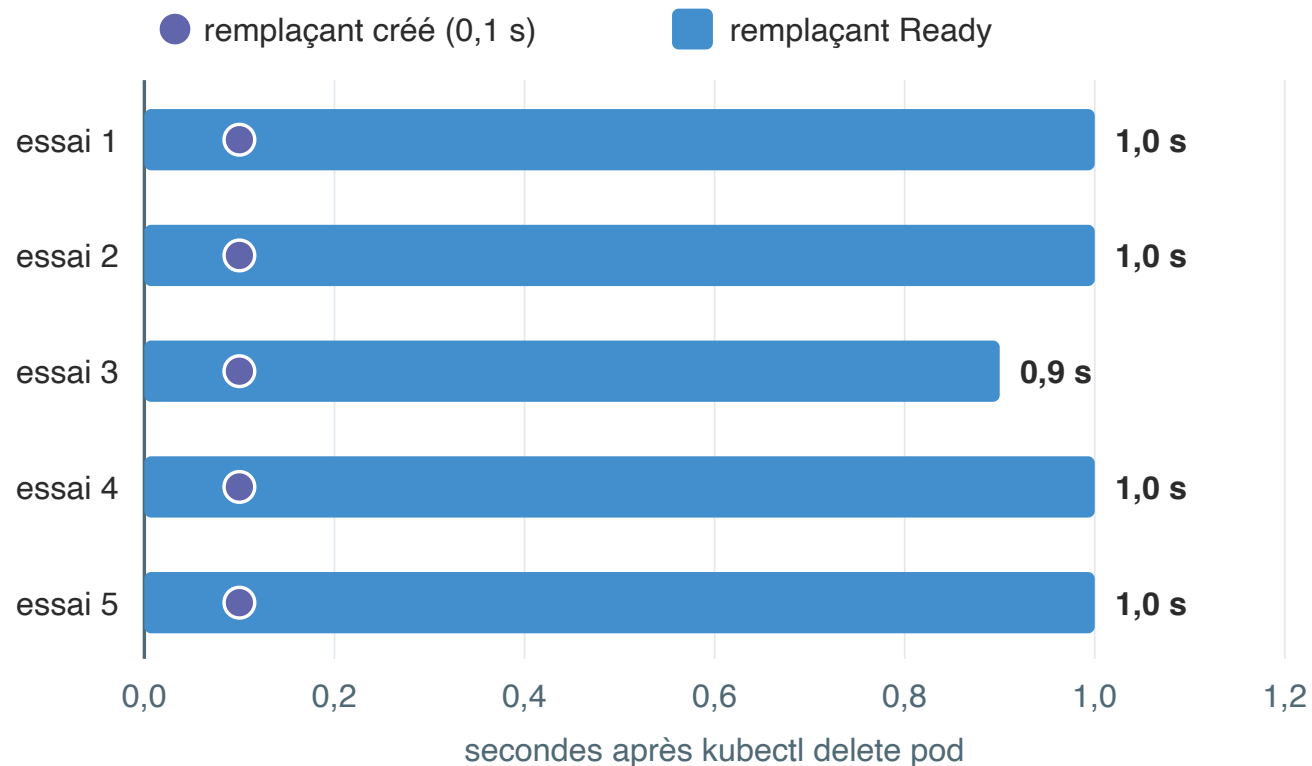


Chaque fonctionnalité a son objet

Fonctionnalité	Objet Kubernetes	Approfondi en
Déploiement automatisé	Deployment, ReplicaSet	M1, M5
Auto-réparation	ReplicaSet, sondes de santé	M1, M5
Mise à l'échelle	replicas, HorizontalPodAutoscaler	M5
Rolling update et rollback	Deployment (révisions)	M5
Découverte de services	Service, DNS interne (CoreDNS)	M1, M5
Configuration	ConfigMap, Secret	M4

Le **Deployment** est le contrôleur que vous utiliserez le plus : il crée un ReplicaSet, qui crée et surveille les Pods.

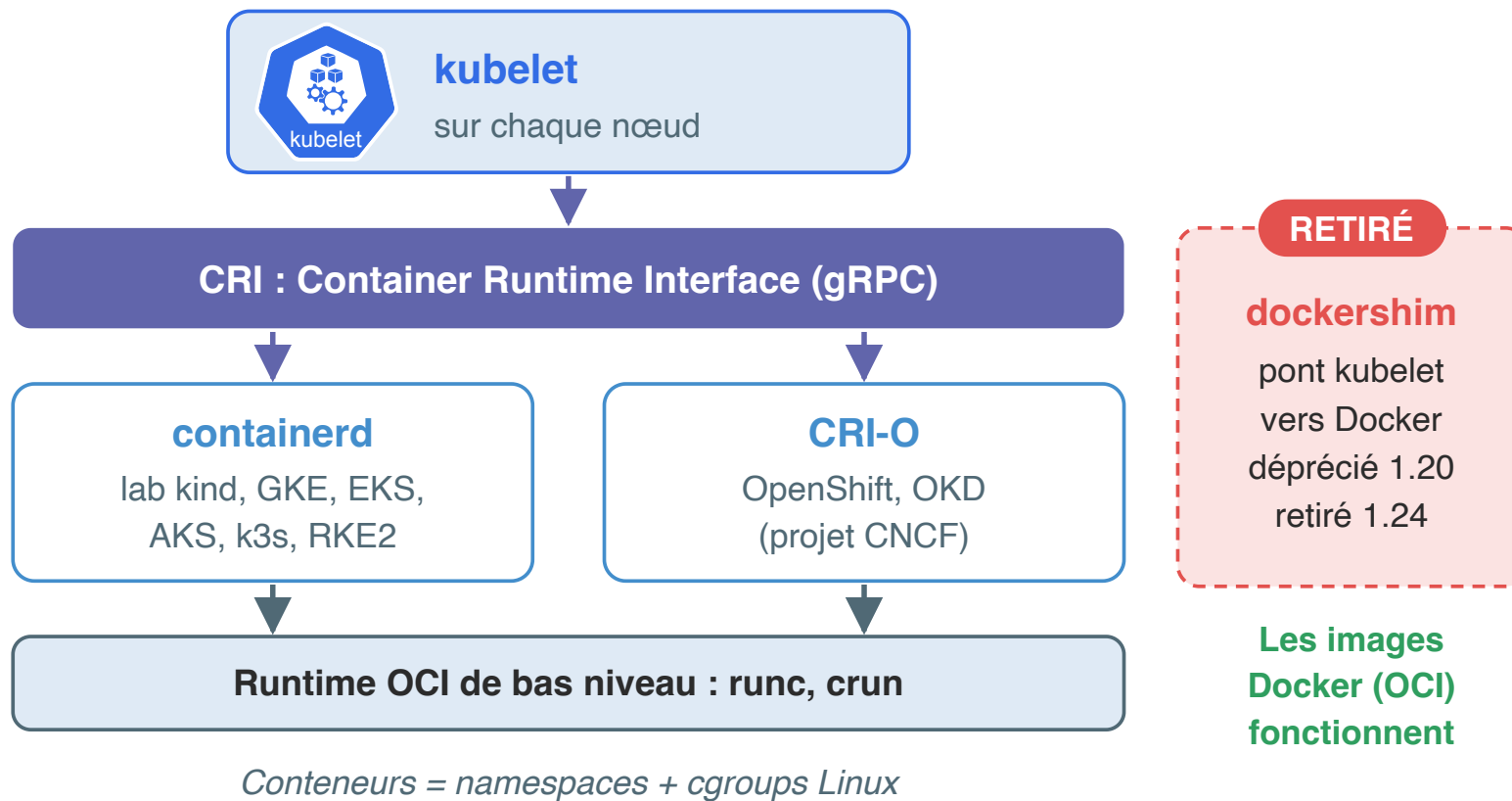
Auto-réparation mesurée sur le lab



Mesuré sur le lab, kind v1.35, 3 nœuds : Deployment de 3 replicas, image en cache.

3. Runtimes et plateformes

Le kubelet parle CRI



La fin de dockershim (v1.24)

Avant la v1.24 (jusqu'en 2022)



dockershim : adaptateur maintenu dans le code du kubelet

Depuis la v1.24 (mai 2022)



Docker reste un excellent outil pour construire les images

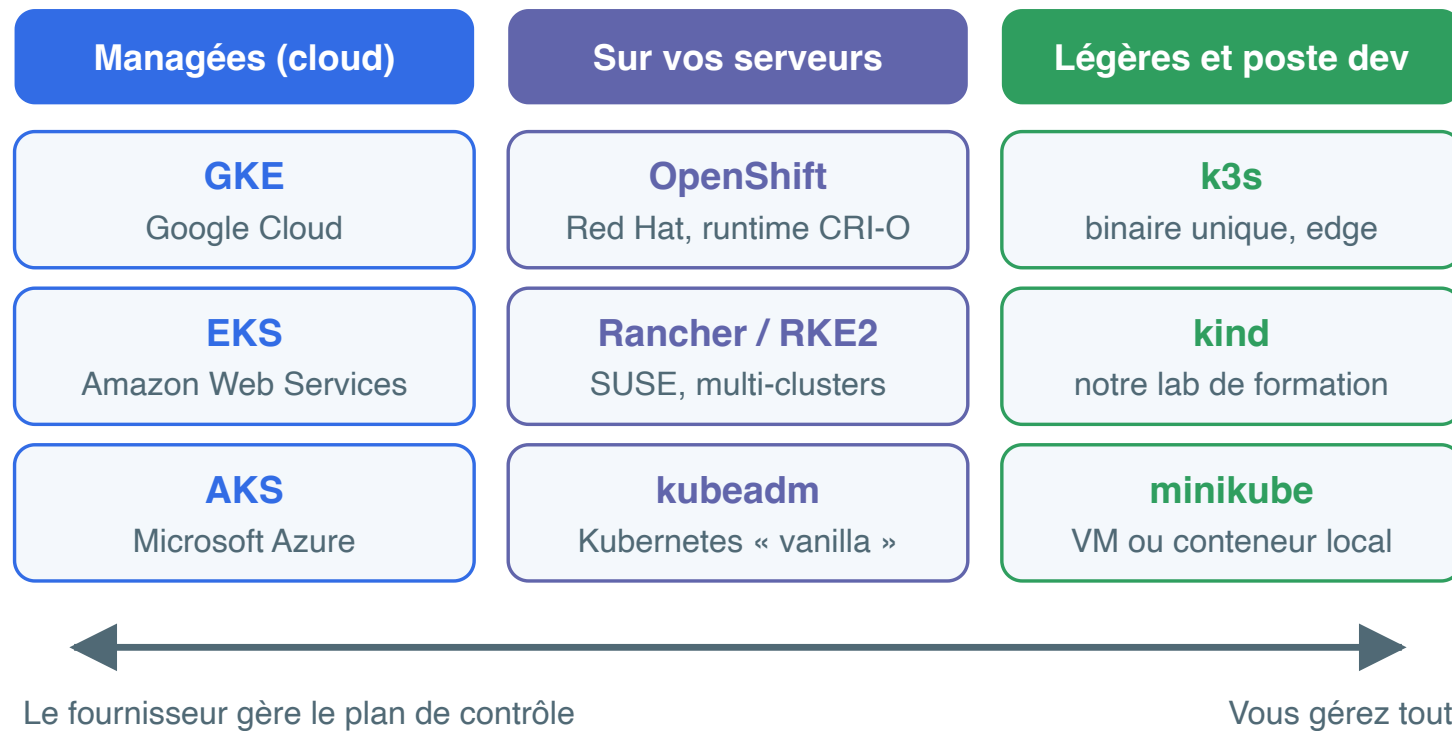
Le runtime de votre lab

```
$ kubectl get nodes -o custom-columns=NOM:.metadata.name,RUNTIME:.status.nodeInfo.containerRuntimeVersion
NOM                                RUNTIME
k8s-formation-control-plane       containerd://2.2.0
k8s-formation-worker              containerd://2.2.0
k8s-formation-worker2             containerd://2.2.0

$ docker exec k8s-formation-worker crictl version      # ou podman exec
RuntimeName: containerd
RuntimeVersion: v2.2.0
RuntimeApiVersion: v1
```

`crictl` dialogue avec le runtime via le socket CRI `unix:///run/containerd/containerd.sock`.

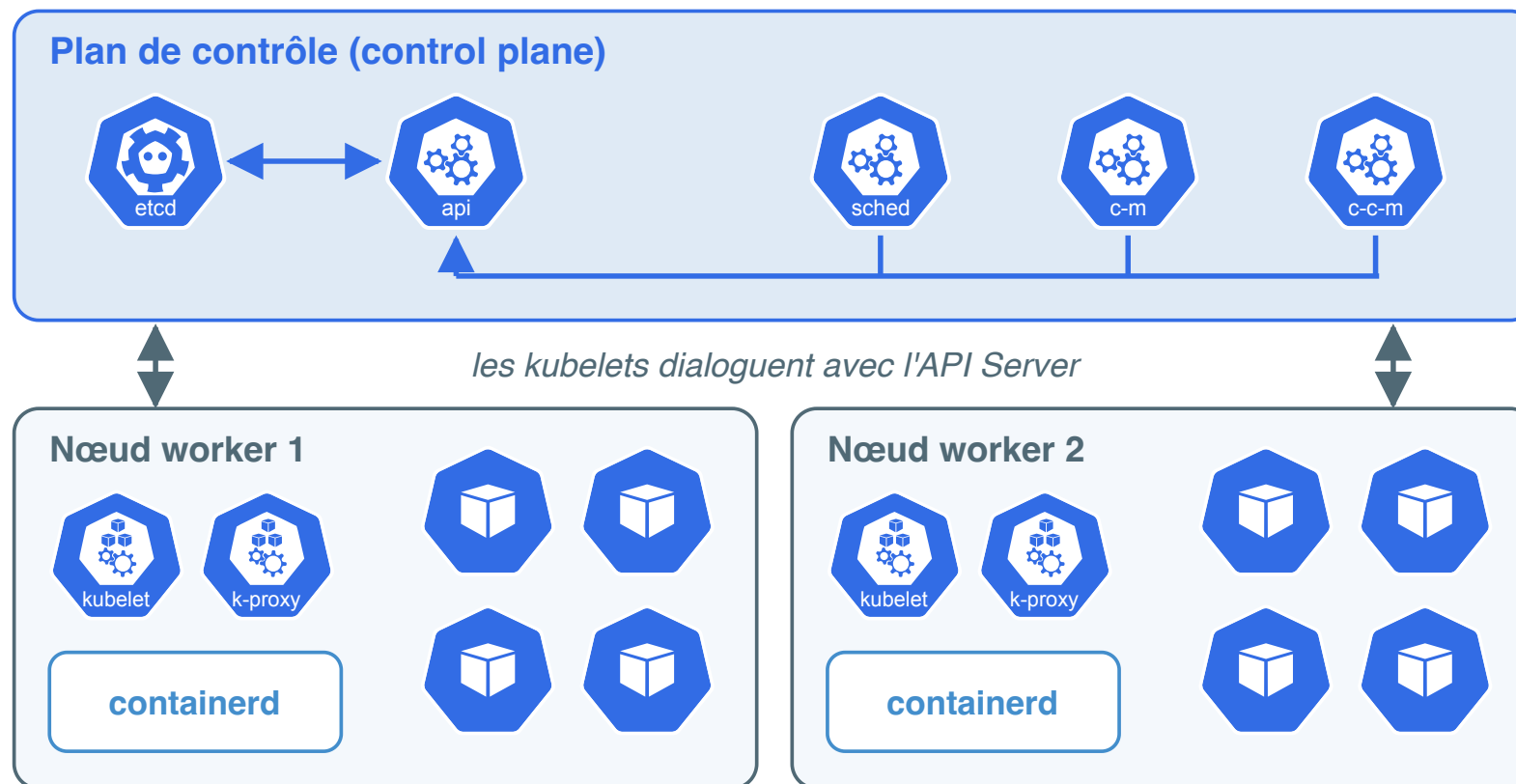
Où trouve-t-on Kubernetes ?



Offres cloud non rejouées sur le lab de formation : nous utilisons kind (module 3).

4. Composants et objets

Les composants en un coup d'œil

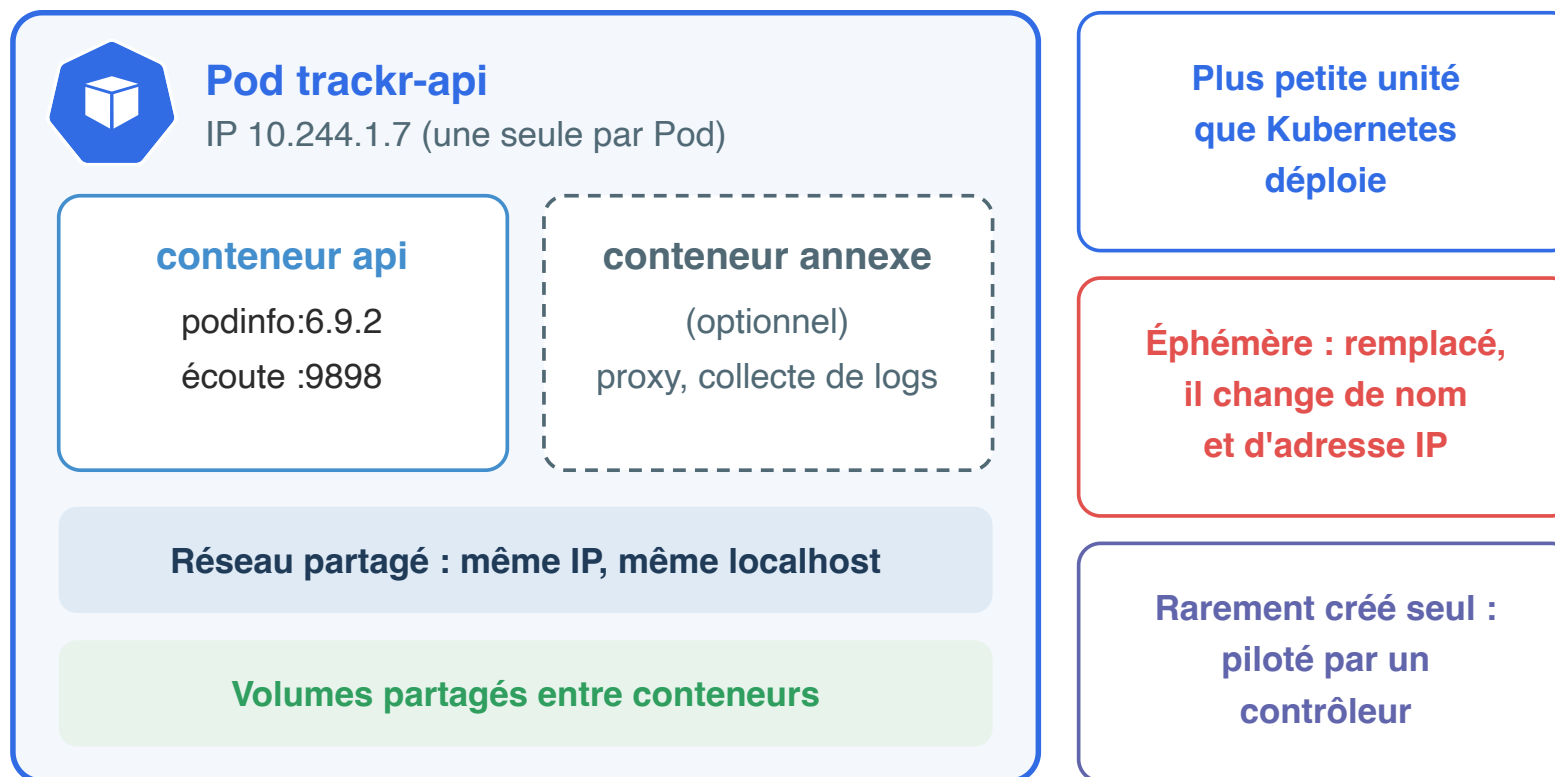


Les composants, vus depuis le lab

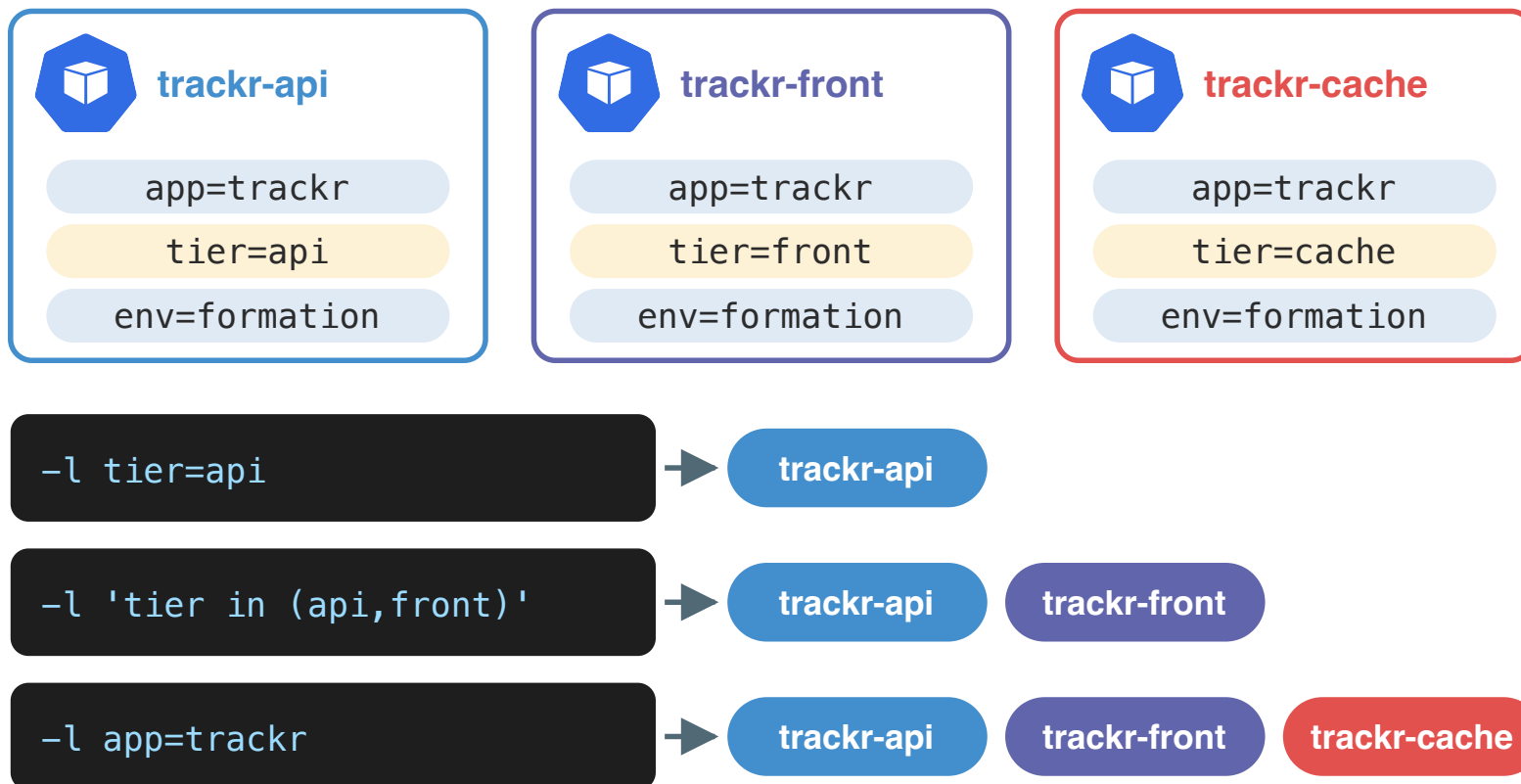
```
$ kubectl get pods -n kube-system -o custom-columns=NAME:.metadata.name,STATUS:.status.phase,NODE:.spec.nodeName
NAME                                STATUS    NODE
coredns-7d764666f9-vdwcd           Running   k8s-formation-control-plane
etcd-k8s-formation-control-plane    Running   k8s-formation-control-plane
kindnet-mkpmj                       Running   k8s-formation-worker
kube-apiserver-k8s-formation-control-plane Running   k8s-formation-control-plane
kube-controller-manager-k8s-formation-control-plane Running   k8s-formation-control-plane
kube-proxy-4hqhg                    Running   k8s-formation-worker
kube-scheduler-k8s-formation-control-plane Running   k8s-formation-control-plane
...
```

Le plan de contrôle tourne lui-même en Pods sur le nœud `control-plane`. Le kubelet et containerd sont des services du nœud : ils n'apparaissent pas ici. Détail au module 2.

Le Pod, plus petite unité déployable



Labels et sélecteurs



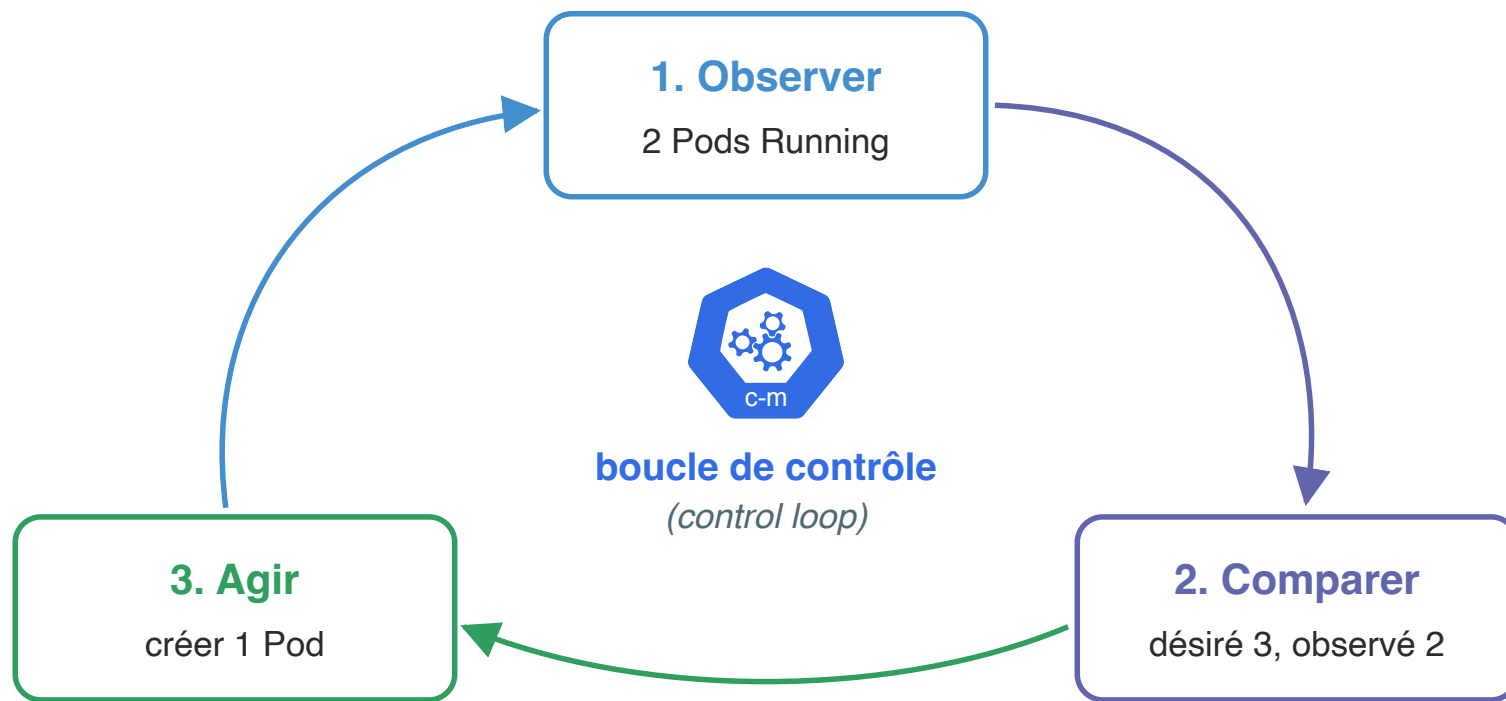
Sélecteurs en pratique

```
$ kubectl get pods -n trackr --show-labels
NAME          READY   STATUS    RESTARTS   AGE   LABELS
trackr-api    1/1     Running   0           32s   app=trackr,env=formation,tier=api
trackr-cache  1/1     Running   0           1s    app=trackr,env=formation,tier=cache
trackr-front  1/1     Running   0           1s    app=trackr,env=formation,tier=front

$ kubectl get pods -n trackr -l 'tier in (api,front)'
NAME          READY   STATUS    RESTARTS   AGE
trackr-api    1/1     Running   0           32s
trackr-front  1/1     Running   0           1s
```

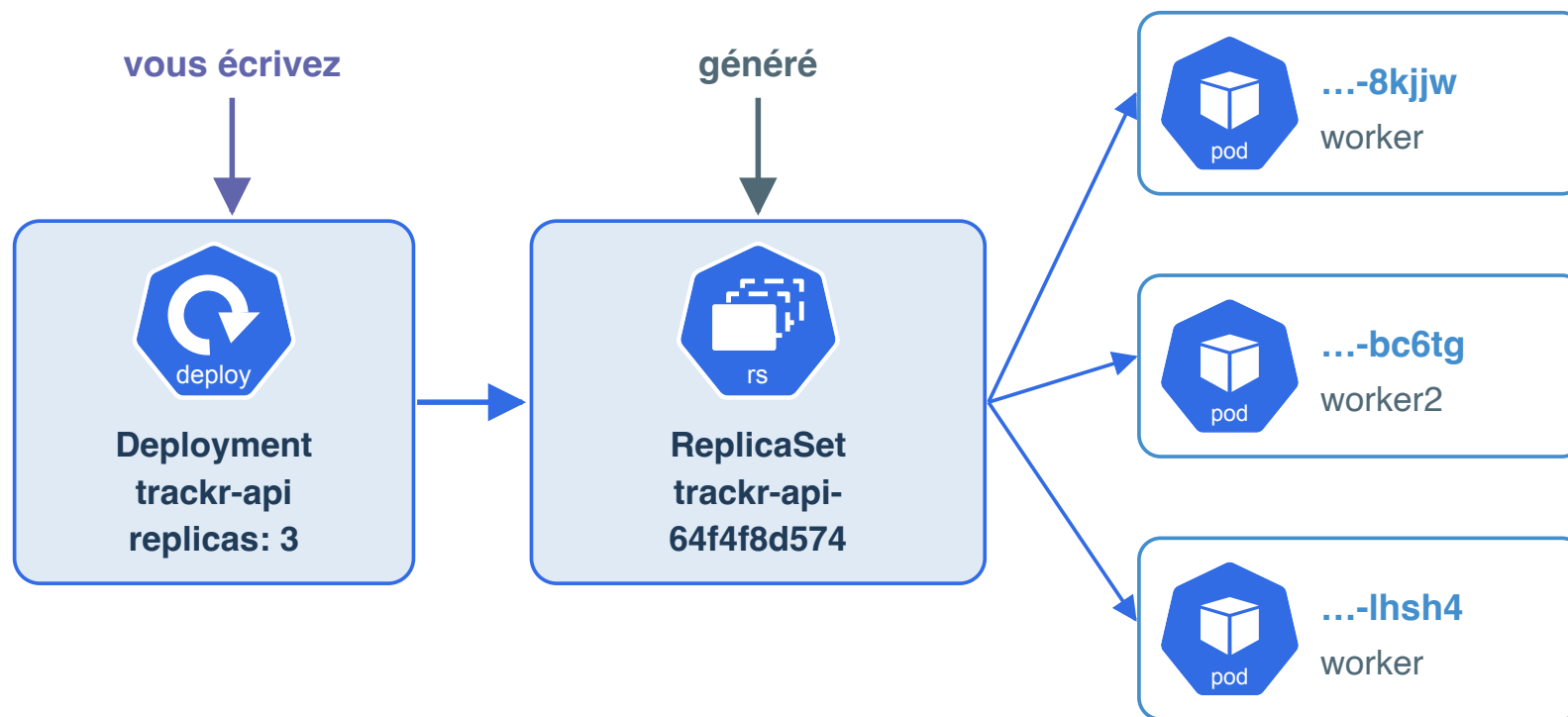
Égalité (=, !=) ou ensemble (in, notin, !clé) : les mêmes règles servent aux Services et aux contrôleurs.

Les contrôleurs : une boucle sans fin



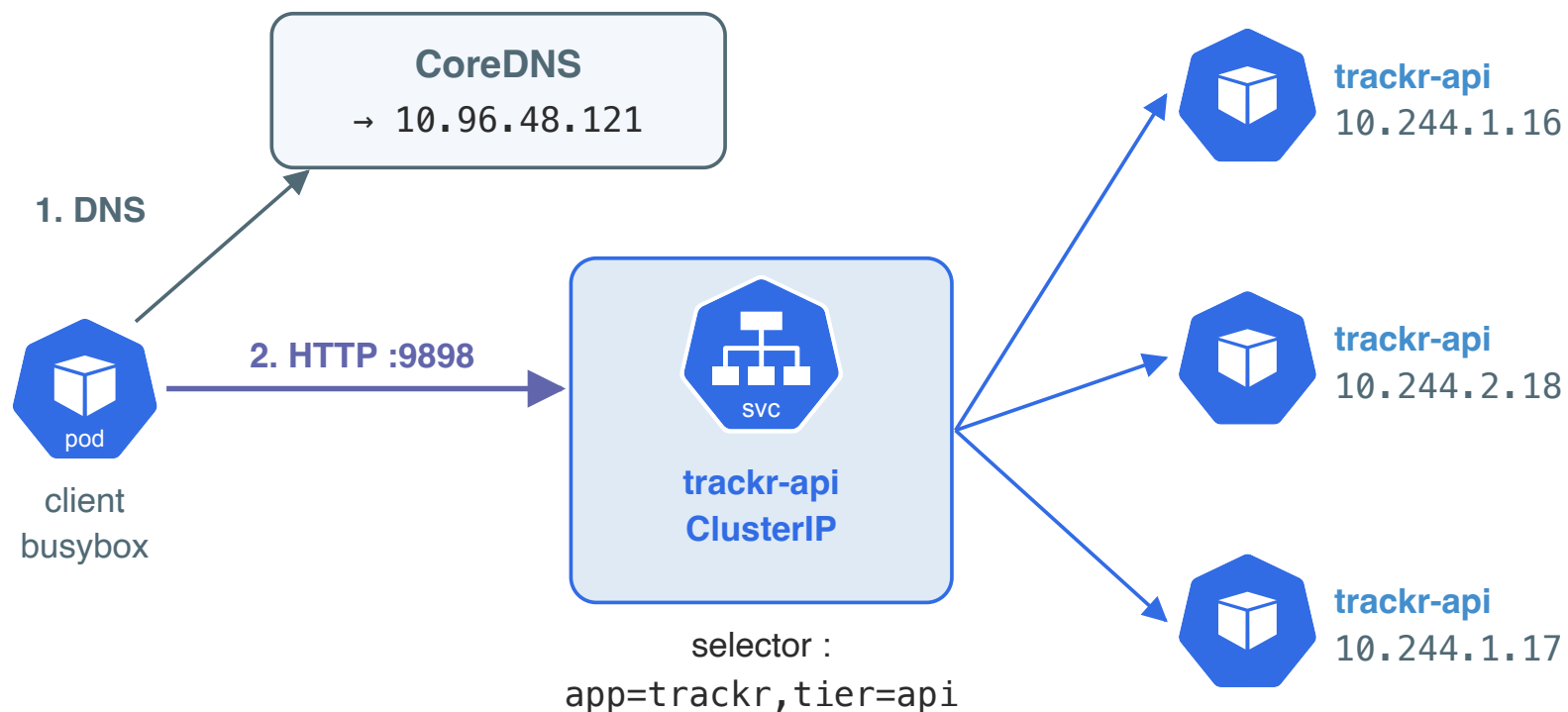
Répétée en permanence par chaque contrôleur

Deployment, ReplicaSet, Pods



Chaque objet désigne son propriétaire (ownerReferences)

Le Service : une adresse stable



Le Service choisit ses Pods par labels, jamais par nom ni par IP

Un Service suit les labels, pas les noms

```
$ kubectl label pod trackr-api -n trackr tier=maintenance --overwrite
$ kubectl get endpointslices -n trackr
NAME                ADDRESSTYPE  PORTS    ENDPOINTS  AGE
trackr-api-sxwtn    IPv4         <unset>  <unset>    9s

$ kubectl label pod trackr-api -n trackr tier=api --overwrite
$ kubectl get endpointslices -n trackr
NAME                ADDRESSTYPE  PORTS    ENDPOINTS  AGE
trackr-api-sxwtn    IPv4         9898     10.244.1.7 12s
```

Retirer un label suffit à sortir un Pod du trafic, sans le supprimer : pratique pour isoler un Pod à diagnostiquer.

5. Déclaratif et kubectl

Impératif ou déclaratif ?

Impératif : comment faire

```
$ kubectl run ...  
$ kubectl label ...  
$ kubectl scale ...
```

- rapide pour tester, débarrer
- état = somme des commandes
- difficile à rejouer, à relire

Déclaratif : quoi obtenir

```
kind: Deployment  
spec:  
  replicas: 3  
  template: ...
```

- fichier versionné dans Git
- apply idempotent, diff, revue
- le cluster converge vers le fichier

De la commande au manifeste

```
kubectl run trackr-api \  
  --image=ghcr.io/stefanprodan/podinfo:6.9.2 \  
  --port=9898 \  
  --dry-run=client -o yaml > pod.yaml
```

`--dry-run=client -o yaml` génère le manifeste sans rien créer : on le complète (labels), puis on l'applique.

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: trackr-api  
  labels: {app: trackr, tier: api, env: formation}  
spec:  
  containers:  
    - name: api  
      image: ghcr.io/stefanprodan/podinfo:6.9.2  
      ports:  
        - containerPort: 9898
```

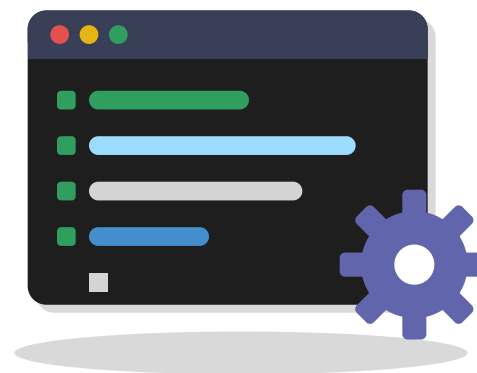
apply, diff : le cluster converge

```
$ kubectl scale deployment trackr-api -n trackr --replicas=5
deployment.apps/trackr-api scaled

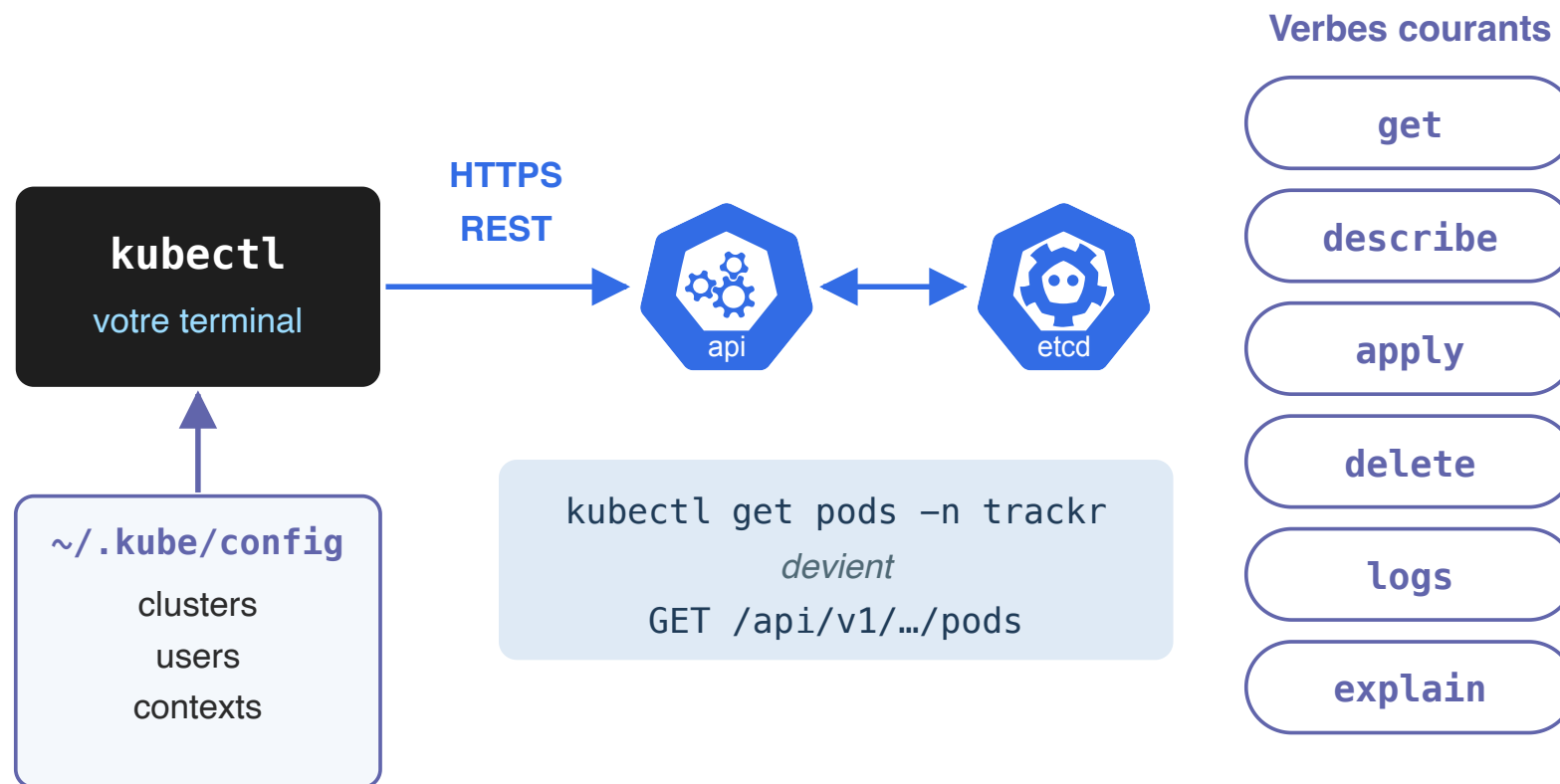
$ kubectl diff -n trackr -f 03-deploy-trackr-api.yaml | grep replicas
- replicas: 5
+ replicas: 3

$ kubectl apply -n trackr -f 03-deploy-trackr-api.yaml
deployment.apps/trackr-api configured
```

Une modification impérative crée une **dérive** ; le fichier versionné la corrige.



kubectl : premier contact



Démo et TP

Démo 1 : premier Pod TrackR

`demos/demo-01-premier-pod.md`

Pod `trackr-api`, labels et sélecteurs, Service, Deployment de 3 replicas et auto-réparation.

TP 1 : trackr-front en recette

`tp/tp-01-premiers-pas.md`

À partir d'une fiche de besoin Logivia, vous écrivez vous-même les manifestes du front, puis vous expliquez ce que le Service et le Deployment font d'un Pod intrus. Durée : 30 min.

À retenir

- Kubernetes **orchestre** : placer, surveiller, remplacer.
- Héritier de Borg, gouverné par la CNCF.
- Le kubelet parle **CRI** ; plus de dockershim depuis v1.24.
- **Pod, labels, sélecteurs** : les briques de base.
- Les **contrôleurs** réconcilient observé et désiré.
- Décrire l'état désiré, puis `kubectl apply`.

